

SIEM Home Lab: Detecting Simulated Attacks with Sysmon, ELK, and Atomic Red Team

Overview

I built a home SIEM lab using Sysmon, ELK, and Atomic Red Team to practice detection, log filtering, simulating real attacker techniques, and writing detection rules to detect potential malicious events. I built this lab to simulate what SOC and Cybersecurity analysts may have to do in their day-to-day lives.

Environment

- Hypervisor: VirtualBox
- Target: Windows 11 Enterprise (Evaluation), standalone/workgroup
- SIEM: Elastic Stack (Elasticsearch + Kibana) — version: 9.5.2
- Log source: Sysmon (SwiftOnSecurity config) + Winlogbeat
- Attack simulation: Atomic Red Team (Invoke-AtomicRedTeam)
- Network: VirtualBox NAT (VM → host via 10.0.2.2)

Architecture / Data flow

Sysmon logs → Winlogbeat → Elasticsearch → Kibana

Sysmon detects and logs system activity (like the Atomic Red Team tests) on the VM.

Winlogbeat reads those logs and ships them over the network to Elasticsearch, which stores and indexes the data. Kibana then queries Elasticsearch to display and search that data.

Detection 1: Suspicious PowerShell Execution

- **Technique:** T1059.001 – PowerShell (Atomic Test #17)
- **What it does:** Detects when a PowerShell Command is executed using an encoded command flag
- **Detection query:** `event.code:1 and process.name:"powershell.exe" and process.args:"-e"`
- **Why this pattern matters:** It detects when a command is executed in the PowerShell using the encoded flag which is meant to bypass traditional security controls
- **Confidence level:** High

- **Why high confidence:** Confidence is high because this combination of behaviors rarely occurs during normal use. Users and admins typically don't need the encoded command flag — it exists primarily to bypass security controls and hide command content from casual inspection, which isn't necessary for someone legitimately operating their own system. Attackers and malicious scripts use it specifically to obscure what a command is doing from both the user and security tooling. The fact that PowerShell was launched by cmd.exe, rather than opened directly, adds further suspicion — a user wanting to run a PowerShell command would simply open PowerShell themselves rather than launching it indirectly through Command Prompt.

Detection 2: System Information Discovery

- **Technique:** T1082 – System Information Discovery (Atomic Test #1)
- **What it does:** Detects whether a systeminfo.exe is called upon
- **Detection query:** `event.code:1 and process.name:"systeminfo.exe"`
- **Confidence level:** Low
- **Why low confidence:** It is difficult to detect whether it is routine check or malicious
- **How you'd improve it in production:** Change detection query to detect where the log is coming from

Detection 3: Registry Run Key Persistence

- **Technique:** T1547.001 – Registry Run Keys (Atomic Test #1)
- **What it does:** Attacker alters the Run Key
- **Detection query:** `event.code:"13" and registry.path:*CurrentVersion\\Run*`
- **Confidence level:** Medium
- **Why medium confidence:** Confidence is medium rather than low because writes to this specific registry path happen far less often than routine commands like systeminfo, but it's not high confidence because the query can't distinguish a legitimate software install from a malicious persistence mechanism, both would trigger the same alert.

Challenges & troubleshooting

Some challenges and troubleshooting that occurred when completing this lab includes:

- VM couldn't initially reach Elasticsearch on the host, had to learn that VirtualBox NAT mode requires using 10.0.2.2 (a special gateway address) instead of localhost to reach the host from inside the VM
- Had to add a Windows Firewall inbound rule to allow traffic on port 9200, since it was blocking the VM's connection attempts
- Windows Defender flagged and interfered with Atomic Red Team's test files during download/install — added a folder exclusion, since this is expected behavior when working with attack-simulation tooling (a good reminder of why this work happens in an isolated VM, never on a real host)

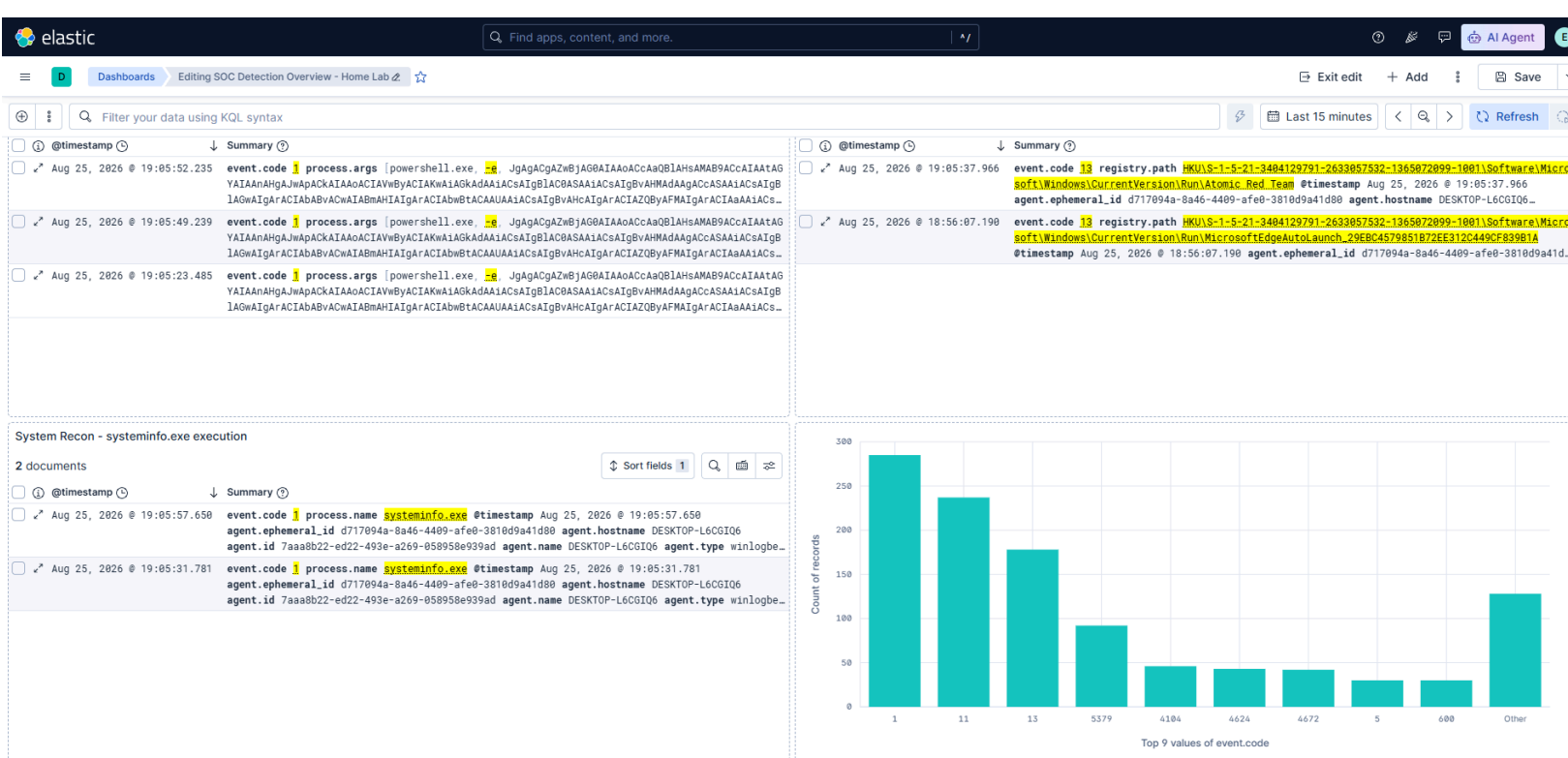


Image of Dashboard with:

- **Top-left panel:** PowerShell encoded-command detection
- **Top-right panel:** Registry Run Key detection
- **Bottom-left panel:** systeminfo detection
- **Bottom-right:** a bonus chart showing event code distribution